

ДЗ_9_АЙКОС

Комкова Полина Дмитриевна БПИ244

Формулировка ДЗ

Побитовая передача целого числа от одного процесса другому с помощью сигналов **SIGUSR1**, **SIGUSR2**. Необходимо написать две программы — передатчик и приемник, которые будут запускаться с разных терминалов. Каждая программа вначале печатает свой **PID** и запрашивает ввод с клавиатуры **PID** программы–собеседника (вводится пользователем). Порядок ввода **PID** не имеет значения.

Программа—передатчик запрашивает с клавиатуры ввод целого десятичного числа (число может быть и отрицательным!). Введенное число побитово передается программе-приемнику с использованием пользовательских сигналов **SIGUSR1** и **SIGUSR2**.

Программа—приемник после завершения приема печатает принятое число в десятичной системе счисления.

Примечание: Каждый бит (0 или 1) передается своим сигналом. Из-за ненадежности сигналов передача последовательности одинаковых битов может приводить к ошибкам. Поэтому каждый новый бит можно передавать только после подтверждения от приемника принятия предыдущего с помощью одного из пользовательских сигналов. Отсутствие подтверждения – грубая ошибка! Завершение передачи можно осуществить, например, передачей сигнала **SIGINT**. Примеры работы можно посмотреть на демонстрационных видео, размещенных в разделе семинара по сигналам.

Исходный код

```
// sender.c
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>
#include <sys/types.h>
```

```

volatile sig_atomic_t ack_received = 0; // Флаг подтверждения

// Обработчик подтверждения от приемника
void handle_ack(int sig) {
    (void)sig;
    ack_received = 1;
}

int main(void) {
    printf("Sender pid = %d\n", (int)getpid()); // Печатаем PID передатчика

    pid_t receiver_pid; // PID приемника
    printf("Input receiver PID: ");
    if (scanf("%d", &receiver_pid) != 1) { // Если ввод некорректен
        perror("scanf");
        return 1;
    }

    int number; // Вводимое число
    printf("Input decimal integer number: ");
    if (scanf("%d", &number) != 1) { // Если ввод некорректен
        perror("scanf");
        return 1;
    }

    signal(SIGUSR1, handle_ack); // Обработчик подтверждения
    unsigned int u = (unsigned int)number; // Преобразуем число в
    неотрицательное

    for (int i = 0; i < 32; ++i) { // Перебираем биты
        ack_received = 0; // Сбрасываем флаг
        int bit = (u >> i) & 1; // Получаем текущий бит
        int sig = bit ? SIGUSR2 : SIGUSR1; // Выбираем сигнал

        if (kill(receiver_pid, sig) == -1) { // Пытаемся отправить сигнал
            perror("kill");
            return 1;
        }

        putchar(bit ? '1' : '0'); // Печатаем бит
        fflush(stdout); // Обновляем вывод

        // Ждем, пока приемник не подтвердит
        while (!ack_received) {
            pause(); // Приостанавливаем выполнение
        }
    }
}

```

```

    }
}

putchar('\n');
if (kill(receiver_pid, SIGINT) == -1) { // Отправляем сигнал
    perror("kill SIGINT");
    return 1;
}

printf("Result = %d\n", number);
return 0;
}

```

```

// receiver.c
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>
#include <sys/types.h>

volatile sig_atomic_t received = 0; // Принятые биты в порядке их прихода
volatile sig_atomic_t bit_count = 0; // Количество принятых бит
volatile sig_atomic_t done = 0; // Флаг завершения
pid_t transmitter_pid = 0; // PID отправителя

// Функция для корректного асинхронного вывода символа
void safe_write_char(char c) {
    write(1, &c, 1);
}

// Обработчик сигнала 1 (для 0)
void handle_sigusr1(int sig) {
    (void)sig;
    bit_count++; // Плюс принятый бит
    if (transmitter_pid) { // Если есть PID отправителя
        kill(transmitter_pid, SIGUSR1); // Отправляем подтверждение
    }
    safe_write_char('0'); // Печатаем 0
}

// Обработчик сигнала 2 (для 1)
void handle_sigusr2(int sig) {
    (void)sig;
    received |= (1u << bit_count); // Ставим 1 в соответствующий бит
    bit_count++; // Плюс принятый бит
}

```

```

    if (transmitter_pid) {
        kill(transmitter_pid, SIGUSR1);
    }
    safe_write_char('1');
}

// Обработчик SIGINT
void handle_sigint(int sig) {
    (void)sig;
    done = 1; // Ставим флаг завершения
}

int main(void) {
    printf("Receiver pid = %d\n", (int)getpid());

    printf("Input sender PID: ");
    if (scanf("%d", &transmitter_pid) != 1) {
        perror("scanf");
        return 1;
    }

    // Ставим обработчики
    signal(SIGUSR1, handle_sigusr1);
    signal(SIGUSR2, handle_sigusr2);
    signal(SIGINT, handle_sigint);

    // Ожидаем завершения
    while (!done) {
        pause();
    }

    putchar('\n');
    int final_value = (int)received; // Преобразуем полученные биты в число
    printf("Result = %d\n", final_value); // Печатаем результат
    return 0;
}

```

Алгоритм работы

1. Есть два процесса: **sender** и **receiver**, они запускаются в разных терминалах
2. Каждый процесс печатает свой PID и просит ввести PID другого процесса. Порядок запуска не важен
3. Передатчик считывает целое число и передает его побитно

- бит 0 передается сигналом SIGUSR1 , бит 1 передается SIGUSR2
 - После отправки каждого бита передатчик ждет подтверждения от приемника, приемник отвечает SIGUSR1
4. После передачи передатчик посылает приемнику SIGINT в качестве конца передачи. Приемник получает сигнал, завершает прием и печатает восстановленное целое число
5. Для побитового восстановления отрицательных чисел я использую двоичное представление в доп коде
- Я передаю unsigned int бит за битом, а затем привожу обратно к int

Примечание

Для реализации на 8 баллов:

- Сигналы по природе своей **асинхронны**
- Также **обработчики сигналов** (handle_sigusr1 , handle_sigusr2 , handle_sigint) выполняются асинхронно по отношению к основному потоку программы
- **Межпроцессное взаимодействие** через kill() - асинхронная отправка сигналов

Пример использования

- Число -2047 (из записи лектора)

```
polia@LAPTOP-KJ14OKRT: ~/AKOC/DZ_9
polia@LAPTOP-KJ14OKRT:~/AKOC/DZ_9$ gcc sender.c -o sender
polia@LAPTOP-KJ14OKRT:~/AKOC/DZ_9$ gcc receiver.c -o receiver
polia@LAPTOP-KJ14OKRT:~/AKOC/DZ_9$ ./sender
Sender pid = 87306
Input receiver PID: 87313
Input decimal integer number: -2047
100000000011111111111111111111111
Result = -2047
polia@LAPTOP-KJ14OKRT:~/AKOC/DZ_9$

polia@LAPTOP-KJ14OKRT: ~/AKOC/DZ_9
polia@LAPTOP-KJ14OKRT:~/AKOC/DZ_9$ ./receiver
Receiver pid = 87313
Input sender PID: 87306
100000000011111111111111111111111
Result = -2047
polia@LAPTOP-KJ14OKRT:~/AKOC/DZ_9$
```

- Число 10

```
polia@LAPTOP-KJ14OKRT: ~/AKOC/DZ_9
polia@LAPTOP-KJ14OKRT:~/AKOC/DZ_9$ ./sender
Sender pid = 2687
Input receiver PID: 2694
Input decimal integer number: 10
01010000000000000000000000000000
Result = 10
polia@LAPTOP-KJ14OKRT:~/AKOC/DZ_9$

polia@LAPTOP-KJ14OKRT: ~/AKOC/DZ_9
polia@LAPTOP-KJ14OKRT:~/AKOC/DZ_9$ ./receiver
Receiver pid = 2694
Input sender PID: 2687
01010000000000000000000000000000
Result = 10
polia@LAPTOP-KJ14OKRT:~/AKOC/DZ_9$
```

Структура файла

HW_9/

— sender.c	# Передатчик
— receiver.c	# Приемник
— README.md	# Описание программы

Компиляция

- *Запускаем 2 терминала.* Первый - для работы **передатчик**

```
gcc sender.c -o sender
```

- Второй - для **приемника**

```
gcc receiver.c -o receiver
```

Тестирование

- Сначала запускаем **передатчик**

```
./sender
```

- Потом - **приемник**

```
./receiver
```

- В полях ввода PID вводим соответствующие id. Для приемника - PID передатчика и наоборот
- В терминале, где запущен передатчик, вводим целое число и получаем вывод двоичного представления числа в обоих терминалах